Intro to DevOps — FULL Exam Answers (DO180/DO188 · LO4–LO6)

> Every item = **Q** (the question) · **DO** (apply this) · **VERIFY** (screenshot this) · **WHY** (the explanation asked for).

> CTRL-F a phrase from the question, or a tag like `{L4.7}`. Replace `<pod>` with the real pod name from `kubectl get pods`.

> Namespace assumed `default`; add `-n NS` otherwise. On OpenShift, `oc` = `kubectl` (see translation table).

::: CHEATSHEET ::: cheatsheet

```
kubectl get pods -o wide -l app=web        # list + node/IP
kubectl get pods,rs,deploy,svc             # overview
kubectl describe pod <pod>                 # events at bottom
kubectl logs <pod> [-c CTR] [--previous] [-f]
kubectl exec -it <pod> [-c CTR] -- sh
kubectl get <obj> <name> -o yaml           # full live spec
kubectl get <obj> <name> -o jsonpath='{.path}'; echo
kubectl apply -f file.yaml                 # create/update from manifest
kubectl delete -f file.yaml
kubectl rollout status|history|undo deployment/web
# GENERATE clean YAML to edit (key exam trick):
kubectl create deployment web --image=nginx --dry-run=client -o yaml > web.yaml
kubectl run p --image=nginx --dry-run=client -o yaml > pod.yaml
```

::: kubectl ↔ oc ::: oc-translate

| Task | kubectl | oc |

|---|---|---|

| switch project | `config set-context --current --namespace=x` | `oc project x` |

| new project | `create namespace x` | `oc new-project x` |

| build+deploy | — | `oc new-app IMAGE~GITURL` |

| expose externally | `expose` → Service | `oc expose svc/web` → **Route** (URL) |

| list URLs | `get ingress` | `oc get routes` |

| shell in pod | `exec -it P -- sh` | `oc rsh P` |

| debug workload | `kubectl debug` | `oc debug deploy/web` |

| grant role | `create rolebinding` | `oc adm policy add-role-to-user` |

| allow root | (PSA labels) | `oc adm policy add-scc-to-user anyuid -z SA` |

::: DIAGNOSIS ::: diagnosis

`kubectl describe pod <pod>` → read Events bottom-up:

```
Pending            → Insufficient cpu/mem | nodeSelector/affinity | taint | unbound PVC
```

```
ImagePullBackOff   → image typo | bad tag | private registry (need imagePullSecrets)
CrashLoopBackOff   → app exits → kubectl logs <pod> --previous
Completed (loops)  → no long-running process → add a command
OOMKilled          → lastState.reason → raise limits.memory
CreateContainerConfigError → missing configMap/secret key
Init:0/1 stuck     → initContainer failing → logs <pod> -c <init>
Terminating stuck  → finalizer/grace → --grace-period=0 --force (risky)
```

Service returns nothing → `kubectl get endpoints SVC` (empty = label mismatch or no Ready pods).

::: LO4 — Kubernetes practical ::: lo4

{L4.1}

> **Q:** Create a Deployment named web running nginx:1.25 with 3 replicas using a single imperative kubectl create deployment command, then export its generated YAML to a file.

DO

```
kubectl create deployment web --image=nginx:1.25 --replicas=3
kubectl get deployment web -o yaml > web.yaml
# clean version (generate without creating):
kubectl create deployment web --image=nginx:1.25 --replicas=3 --dry-run=client -o yaml > web.yaml
```

VERIFY

```
kubectl get deploy web
kubectl get pods -l app=web
cat web.yaml
```

WHY One imperative `create deployment`. `-o yaml` exports the live object; `--dry-run=client -o yaml` produces a clean manifest without touching the cluster. (Older kubectl: no `--replicas` on create → `kubectl scale deployment web --replicas=3` after.)

{L4.2}

> **Q:** Enable pulling images from DockerHub as an authenticated user to lower the chance of hitting the pull limit. What kind of resource do you need to create?

DO — create a **docker-registry Secret** (type `kubernetes.io/dockerconfigjson`):

```
kubectl create secret docker-registry dockerhub \
  --docker-server=docker.io \
  --docker-username=YOURUSER \
  --docker-password=YOURPASS \
  --docker-email=you@example.com
# attach to the default ServiceAccount (covers all pods in the namespace):
kubectl patch serviceaccount default -p '{"imagePullSecrets":[{"name":"dockerhub"}]}'
```

Per-deployment alternative: add under `spec.template.spec.imagePullSecrets: [{name: dockerhub}]`.

VERIFY

```
kubectl get secret dockerhub -o jsonpath='{.type}'; echo     # kubernetes.io/dockerconfigjson
kubectl get sa default -o jsonpath='{.imagePullSecrets}'; echo
```

WHY The resource is a **Secret** of type docker-registry. Authenticated pulls get a higher DockerHub rate limit than anonymous. Same pattern with `--docker-server=quay.io` for Quay.

{L4.3}

> **Q:** Scale web from 3 to 5 replicas two different ways — once with kubectl scale, once by editing the manifest — and show the ReplicaSet that results.

DO

```
kubectl scale deployment web --replicas=5          # way 1: imperative
kubectl edit deployment web                         # way 2: change spec.replicas to 5, save
# (or edit web.yaml -> spec.replicas: 5 -> kubectl apply -f web.yaml)
```

VERIFY

```
kubectl get deploy web
kubectl get rs -l app=web        # SAME ReplicaSet, DESIRED now 5
kubectl get pods -l app=web
```

WHY Both edit `spec.replicas`. No new ReplicaSet is created because the pod template didn't change — a new RS appears only on a template change (image/env/etc.).

{L4.4}

> **Q:** Perform a rolling update of web from nginx:1.25 to nginx:1.27 and watch it with kubectl rollout status.

DO

```
kubectl set image deployment/web web=nginx:1.27     # 'web' = container name
kubectl rollout status deployment/web
```

VERIFY

```
kubectl get rs -l app=web        # new RS up, old RS scaled to 0
kubectl describe deployment web | grep -i image
```

WHY `set image` patches the pod template → new RS, pods replaced gradually. Confirm the container name with `kubectl get deploy web -o jsonpath='{.spec.template.spec.containers[*].name}'`.

{L4.5}

> **Q:** View a Deployment's rollout history and roll back to the previous revision. Explain what the CHANGE-CAUSE column shows and how to populate it.

DO

```
kubectl rollout history deployment/web
kubectl rollout history deployment/web --revision=2     # detail of one revision
kubectl rollout undo deployment/web                      # back to previous
kubectl rollout undo deployment/web --to-revision=1      # to a specific revision
# populate CHANGE-CAUSE:
kubectl annotate deployment/web kubernetes.io/change-cause="update to nginx:1.27" --overwrite
```

VERIFY

```
kubectl rollout history deployment/web      # CHANGE-CAUSE column now filled
```

WHY CHANGE-CAUSE shows the `kubernetes.io/change-cause` annotation for each revision; it's `<none>` until you set it. (Legacy `--record` is deprecated.)

{L4.6}

> **Q:** Set the update strategy to RollingUpdate with maxSurge: 1 and maxUnavailable: 0, and explain the effect on availability during an update.

DO

```
kubectl patch deployment web -p \
'{"spec":{"strategy":{"type":"RollingUpdate","rollingUpdate":{"maxSurge":1,"maxUnavailable":0}}}}'
```

Manifest form:

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

VERIFY

```
kubectl get deploy web -o jsonpath='{.spec.strategy}'; echo
```

WHY `maxUnavailable: 0` = never drop below the desired count → **zero-downtime**; `maxSurge: 1` = at most one extra pod during the swap, and a new pod must be Ready before an old one is removed. Values may be counts or percentages; both can't be 0.

{L4.7}

> **Q:** Switch a Deployment's strategy to Recreate and describe a concrete scenario where this is required instead of RollingUpdate.

DO — patch an existing Deployment:

```
kubectl patch deployment web -p '{"spec":{"strategy":{"type":"Recreate"}}}'
```

Or full manifest (`web-recreate.yaml`, then `kubectl apply -f web-recreate.yaml`):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: web
spec:
  replicas: 3
  strategy:
    type: Recreate            # <-- note: no rollingUpdate block allowed here
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web
        image: nginx:1.25
        ports:
        - containerPort: 80
```

VERIFY — confirm strategy, then trigger an update and watch ALL old pods die before new ones start:

```
kubectl get deploy web -o jsonpath='{.spec.strategy.type}'; echo     # Recreate
kubectl set image deployment/web web=nginx:1.27
kubectl get pods -l app=web -w     # all 3 Terminating -> 0 -> new 3 ContainerCreating
```

WHY Recreate **terminates every old pod, then creates the new ones** → a brief outage (unlike RollingUpdate). It's **required when the old and new versions cannot run at the same time**, e.g.:

- the app mounts a **ReadWriteOnce** volume that only one pod can hold at a time;

- a **database schema migration** where two app versions writing concurrently would corrupt data;

- a **singleton** that must never have two live instances (leader/lock conflicts).

{L4.8}

> **Q:** Add CPU/memory requests and limits to a Deployment's container and verify they appear in the running pod spec.

DO

```
kubectl set resources deployment web \
  --requests=cpu=100m,memory=128Mi \
  --limits=cpu=500m,memory=256Mi
```

Manifest form (inside the container):

```
resources:
  requests: {cpu: "100m", memory: "128Mi"}
  limits:   {cpu: "500m", memory: "256Mi"}
```

VERIFY

```
kubectl get pod -l app=web -o jsonpath='{.items[0].spec.containers[0].resources}'; echo
kubectl describe pod -l app=web | grep -A4 -iE 'limits|requests'
```

WHY `requests` = the guaranteed amount the scheduler reserves; `limits` = the hard cap (CPU throttled at the limit, memory over the limit → **OOMKilled**). Units: `m` = millicores, `Mi`/`Gi` = mebibytes/gibibytes.

{L4.9}

> **Q:** Set revisionHistoryLimit: 3 and explain how it affects your ability to roll back and how many old ReplicaSets are kept.

DO

```
kubectl patch deployment web -p '{"spec":{"revisionHistoryLimit":3}}'
```

VERIFY

```
kubectl get deploy web -o jsonpath='{.spec.revisionHistoryLimit}'; echo
kubectl get rs -l app=web        # at most 3 old (scaled-to-0) RSs retained
```

WHY Kubernetes keeps the last **3** old ReplicaSets (scaled to 0) for rollback; older ones are garbage-collected, so you can only roll back as far as the retained history. Default is 10; setting `0` disables rollback entirely.

{L4.10}

> **Q:** Set a Deployment's image to a non-existent tag, observe the rollout get stuck, and explain why the old pods keep serving traffic.

DO

```
kubectl set image deployment/web web=nginx:doesnotexist
kubectl rollout status deployment/web        # hangs
kubectl get pods -l app=web                  # new pod ImagePullBackOff / ErrImagePull
kubectl describe pod <new-pod> | tail
# recover:
kubectl rollout undo deployment/web
```

VERIFY

```
kubectl get rs -l app=web      # old RS still has running pods; new RS stuck at 0 ready
```

WHY With RollingUpdate (and `maxUnavailable` protection), a new pod must become **Ready** before an old one is removed. The new pod never pulls its image → never Ready → the **old ReplicaSet keeps running and serving traffic**.

{L4.11}

> **Q:** Use a label selector to list only the pods belonging to one Deployment, and explain the Deployment → ReplicaSet → Pod selector relationship.

DO / VERIFY

```
kubectl get pods -l app=web -o wide
kubectl get deploy web -o jsonpath='{.spec.selector.matchLabels}'; echo
kubectl get rs -l app=web
kubectl describe rs <rs-name> | grep -i selector
```

WHY The Deployment's `selector` matches **ReplicaSets**; each RS's selector matches **Pods** via the labels in `spec.template.metadata.labels`. Chain: Deployment → RS (one per template revision) → Pods. The pod-template labels must satisfy the selector or the manifest is rejected.

{L4.12}

> **Q:** Expose a Deployment with kubectl expose, then explain what object was created and how its selector was derived.

DO

```
kubectl expose deployment web --port=80 --target-port=80
```

VERIFY

```
kubectl get svc web
kubectl get svc web -o jsonpath='{.spec.selector}'; echo    # copied from the deployment's pod labels
kubectl describe svc web
```

WHY It creates a **Service** (default type **ClusterIP**). Its `selector` is **copied from the Deployment's pod-template labels**, so it automatically targets those pods. `--type=NodePort|LoadBalancer` and `--target-port` (the container port) change behavior.

{L4.13}

> **Q:** Add a sidecar (second) container to a Deployment's pod template and explain how the two containers share the pod network and volumes.

DO — pod template with two containers sharing an emptyDir:

```
spec:
  volumes:
  - name: shared
    emptyDir: {}
  containers:
  - name: web
    image: nginx:1.25
    ports: [{containerPort: 80}]
    volumeMounts: [{name: shared, mountPath: /usr/share/nginx/html}]
  - name: sidecar
    image: busybox
    command: ["sh","-c","while true; do date > /data/index.html; sleep 10; done"]
```

```
      volumeMounts: [{name: shared, mountPath: /data}]
```

VERIFY

```
kubectl exec -it <pod> -c sidecar -- cat /data/index.html
kubectl exec -it <pod> -c web -- sh -c 'wget -qO- localhost:80'   # sees the sidecar's file
```

WHY Containers in one pod share the **network namespace** (same pod IP; they reach each other on `127.0.0.1:<port>`; ports must not collide) and any **volume** mounted into both (here the sidecar writes the file nginx serves).

{L4.14}

> **Q:** Write a Deployment manifest for httpd:2.4 with 2 replicas, a named container port, and a nodeSelector; explain why it stays Pending if no node matches.

DO (`httpd.yaml` → `kubectl apply -f httpd.yaml`)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpd
  labels: {app: httpd}
spec:
  replicas: 2
  selector:
    matchLabels: {app: httpd}
  template:
    metadata:
      labels: {app: httpd}
    spec:
      nodeSelector:
        disktype: ssd
      containers:
      - name: httpd
        image: httpd:2.4
        ports:
        - name: http
          containerPort: 80
```

VERIFY

```
kubectl get pods -l app=httpd                       # Pending
kubectl describe pod -l app=httpd | grep -A3 Events  # "didn't match Pod's node affinity/selector"
# fix by labelling a node:
kubectl label node minikube disktype=ssd
kubectl get pods -l app=httpd                       # now Running
```

WHY No node carries the label `disktype=ssd`, so the scheduler has nowhere to place the pods → they stay **Pending** until a matching node exists (or the selector is removed).

{L4.15}

> **Q:** Create a bare Pod (no controller) running busybox that sleeps, then explain what happens when you delete it versus a Deployment-managed pod.

DO

```
kubectl run busy --image=busybox --restart=Never --command -- sleep 3600
```

VERIFY — bare pod is gone for good

```
kubectl get pod busy
kubectl delete pod busy
kubectl get pod busy            # NotFound — nothing recreates it
```

VERIFY — managed pod is recreated

```
kubectl delete pod $(kubectl get pod -l app=web -o jsonpath='{.items[0].metadata.name}')
kubectl get pods -l app=web    # the ReplicaSet immediately spins up a replacement
```

WHY A bare Pod has no controller, so deleting it is permanent. A Deployment-managed pod is recreated by its ReplicaSet to maintain the desired replica count. Bare pods have no self-healing, scaling, or rollouts.

{L4.16}

> **Q:** Write a StatefulSet for redis:7 with 3 replicas plus a headless Service, and show the stable pod names and per-pod DNS records.

DO (`redis.yaml` → `kubectl apply -f redis.yaml`)

```
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  clusterIP: None          # headless
  selector: {app: redis}
  ports:
  - port: 6379
    name: redis
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  serviceName: redis        # must reference the headless Service
  replicas: 3
  selector:
    matchLabels: {app: redis}
  template:
    metadata:
      labels: {app: redis}
    spec:
      containers:
      - name: redis
        image: redis:7
        ports:
        - containerPort: 6379
          name: redis
```

VERIFY

```
kubectl get pods -l app=redis           # redis-0, redis-1, redis-2 (stable, ordinal names)
kubectl get statefulset redis
kubectl run tmp --rm -it --image=busybox -- nslookup redis-0.redis.default.svc.cluster.local
kubectl run tmp --rm -it --image=busybox -- nslookup redis.default.svc.cluster.local   # all pod IPs
```

WHY A StatefulSet gives stable, ordinal names `redis-0/1/2`; the **headless Service** (`clusterIP: None`) publishes a per-pod DNS record `redis-N.redis.<ns>.svc.cluster.local` instead of one virtual IP.

{L4.17}

> **Q:** Create a DaemonSet running a busybox agent and explain why exactly one pod runs per node.

DO (`agent.yaml` → `kubectl apply -f agent.yaml`)

```
apiVersion: apps/v1
kind: DaemonSet
metadata: {name: agent}
spec:
  selector:
    matchLabels: {app: agent}
  template:
```

```
    metadata:
      labels: {app: agent}
    spec:
      containers:
      - name: agent
        image: busybox
        command: ["sh","-c","while true; do sleep 3600; done"]
```

VERIFY

```
kubectl get ds agent
kubectl get pods -l app=agent -o wide     # one pod, one per node
kubectl get nodes
```

WHY The DaemonSet controller schedules **exactly one pod on every eligible node** (and automatically onto new nodes as they join). On single-node minikube that's one pod. Used for node-level agents (logging, monitoring, CNI).

{L4.18}

> **Q:** Create a Job using busybox/perl that computes something once and completes; show COMPLETIONS and read the result from logs.

DO

```
kubectl create job pi --image=perl:5.34.0 -- perl -Mbignum=bpi -wle 'print bpi(2000)'
# simpler busybox alternative:
# kubectl create job calc --image=busybox -- sh -c 'echo $((6*7))'
```

VERIFY

```
kubectl get job pi                              # COMPLETIONS 1/1 when done
kubectl wait --for=condition=complete job/pi --timeout=120s
kubectl logs job/pi                             # the computed result
```

WHY A Job runs its pod(s) to **completion** (process exits 0). `COMPLETIONS` shows succeeded/desired; the result is read from the pod logs. `completions`/`parallelism` control multiple runs.

{L4.19}

> **Q:** Create a CronJob that prints the date every minute; show how to suspend it and how to list the Jobs it spawns.

DO

```
kubectl create cronjob date --image=busybox --schedule="* * * * *" -- date
```

VERIFY / manage

```
kubectl get cronjob date
kubectl get jobs                # spawned jobs named date-<timestamp>
kubectl get pods                # their pods
# suspend (stops creating new Jobs; running ones finish):
kubectl patch cronjob date -p '{"spec":{"suspend":true}}'
kubectl get cronjob date -o jsonpath='{.spec.suspend}'; echo    # true
```

WHY Schedule uses cron syntax (`*` = every minute). `suspend: true` halts new Job creation. Each run creates a Job named `date-<timestamp>` that creates a pod.

{L4.20}

> **Q:** Manually run a Job from an existing CronJob (kubectl create job --from=cronjob/...) to test it on demand.

DO

```
kubectl create job manual-run-1 --from=cronjob/date
```

VERIFY

```
kubectl get job manual-run-1
kubectl logs job/manual-run-1
```

WHY Instantiates the CronJob's job template immediately, without waiting for the schedule — the standard way to test a CronJob on demand.

{L4.21}

> **Q:** Show that a StatefulSet creates and terminates pods in order, and contrast this with a Deployment.

DO / VERIFY

```
# StatefulSet: scale up, watch ordered creation 0 -> 1 -> 2
kubectl scale statefulset redis --replicas=3
kubectl get pods -l app=redis -w        # each becomes Ready before the next starts
# scale down, watch reverse order 2 -> 1 -> 0
kubectl scale statefulset redis --replicas=1
kubectl get pods -l app=redis -w
# Deployment for contrast: all at once, random names
kubectl scale deployment web --replicas=5
kubectl get pods -l app=web -w
```

WHY A StatefulSet creates `0→1→2` (each must be Ready first) and terminates in **reverse** `2→1→0`, with stable ordinal names. A Deployment creates/terminates pods **in parallel**, with random hash-suffix names and no ordering guarantee.

{L4.22}

> **Q:** Create a multi-container Pod where two containers share an emptyDir — one writes a file, the other reads it. Prove it's shared.

DO (`shared.yaml` → `kubectl apply -f shared.yaml`)

```
apiVersion: v1
kind: Pod
metadata: {name: shared-demo}
spec:
  volumes:
  - name: shared
    emptyDir: {}
  containers:
  - name: writer
    image: busybox
    command: ["sh","-c","echo hello-from-writer > /data/msg; sleep 3600"]
    volumeMounts: [{name: shared, mountPath: /data}]
  - name: reader
    image: busybox
    command: ["sh","-c","sleep 3600"]
    volumeMounts: [{name: shared, mountPath: /data}]
```

VERIFY

```
kubectl exec shared-demo -c reader -- cat /data/msg     # hello-from-writer
```

WHY The same `emptyDir` volume is mounted into both containers, so the file the writer creates is visible to the reader — proving shared pod-scoped scratch storage.

{L4.23}

> **Q:** List the StorageClasses, PVs and PVCs.

DO / VERIFY

```
kubectl get storageclass          # or: kubectl get sc
kubectl get pv
kubectl get pvc --all-namespaces
```

WHY StorageClasses define how storage is dynamically provisioned; PVs are the actual volumes; PVCs are pods' claims that bind to PVs. On minikube the default class is usually `standard` (hostpath provisioner).

{L4.24}

> **Q:** Mount a ConfigMap as a volume so each key becomes a file, and verify the contents inside the pod.

DO

```
kubectl create configmap appcfg --from-literal=color=blue --from-literal=mode=dark
```

Pod (`cm-vol.yaml`):

```
apiVersion: v1
kind: Pod
metadata: {name: cm-demo}
spec:
  volumes:
  - name: cfg
    configMap: {name: appcfg}
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","sleep 3600"]
    volumeMounts:
    - name: cfg
      mountPath: /etc/cfg
```

VERIFY

```
kubectl exec cm-demo -- ls /etc/cfg          # color  mode
kubectl exec cm-demo -- cat /etc/cfg/color    # blue
```

WHY Mounting a ConfigMap as a volume projects **each key as a file** (filename = key, contents = value) under the mountPath.

{L4.25}

> **Q:** Mount a single ConfigMap key to a specific path using subPath; explain when it's needed and its update caveat.

DO

```
    volumeMounts:
    - name: cfg
      mountPath: /etc/app/color.conf
      subPath: color
  volumes:
  - name: cfg
    configMap: {name: appcfg}
```

VERIFY

```
kubectl exec <pod> -- cat /etc/app/color.conf    # blue
kubectl exec <pod> -- ls /etc/app                 # existing files NOT hidden
```

WHY `subPath` injects **one** key as a single file **without masking** other files already in that directory (a normal volume mount would replace the whole directory). **Caveat:** subPath mounts do **not** receive live updates when the ConfigMap changes — you must restart the pod.

{L4.26}

> **Q:** Mount a Secret as a volume and verify the files contain decoded values with restrictive permissions.

DO

```
kubectl create secret generic mysecret --from-literal=username=admin --from-literal=password=s3cret
  volumes:
  - name: sec
    secret:
      secretName: mysecret
      defaultMode: 0400
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","sleep 3600"]
    volumeMounts:
    - name: sec
      mountPath: /etc/sec
      readOnly: true
```

VERIFY

```
kubectl exec <pod> -- ls -l /etc/sec        # files show -r-------- (0400)
kubectl exec <pod> -- cat /etc/sec/password # s3cret (decoded, not base64)
```

WHY A Secret mounted as a volume exposes each key as a file with **decoded** contents on a tmpfs (RAM) mount; `defaultMode: 0400` makes them owner-read-only.

{L4.27}

> **Q:** Show that scaling a StatefulSet creates one PVC per replica, then explain what happens to those PVCs when the StatefulSet is deleted.

DO — add a volumeClaimTemplate to the redis StatefulSet:

```
spec:
  # ...serviceName, replicas, selector, template as in {L4.16}...
  volumeClaimTemplates:
  - metadata: {name: data}
    spec:
      accessModes: [ReadWriteOnce]
      resources: {requests: {storage: 1Gi}}
```

VERIFY

```
kubectl get pvc                       # data-redis-0, data-redis-1, data-redis-2
kubectl delete statefulset redis
kubectl get pvc                       # PVCs STILL EXIST (data preserved)
kubectl delete pvc -l app=redis       # manual cleanup to reclaim storage
```

WHY Each replica gets its own PVC named `<template>-<pod>` (e.g. `data-redis-0`). When the StatefulSet is deleted, the **PVCs are retained by default** so data survives a recreate — you must delete them manually. (Newer k8s adds `persistentVolumeClaimRetentionPolicy` to change this.)

{L4.28}

> **Q:** Use an initContainer to pre-populate data into a shared volume before the main container reads it.

DO (`init.yaml`)

```
apiVersion: v1
kind: Pod
metadata: {name: init-demo}
spec:
  volumes:
  - name: data
    emptyDir: {}
  initContainers:
  - name: seed
    image: busybox
    command: ["sh","-c","echo seeded-data > /data/file"]
    volumeMounts: [{name: data, mountPath: /data}]
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","cat /data/file; sleep 3600"]
    volumeMounts: [{name: data, mountPath: /data}]
```

VERIFY

```
kubectl logs init-demo -c app           # prints seeded-data
kubectl exec init-demo -c app -- cat /data/file
```

WHY Init containers run to completion **before** the app container starts, so the data they write to the shared volume is guaranteed present when the main container reads it.

{L4.29}

> **Q:** Set readOnly: true on a volume mount and prove writes are rejected.

DO

```
    volumeMounts:
    - name: data
      mountPath: /data
      readOnly: true
  volumes:
  - name: data
    emptyDir: {}
```

VERIFY

```
kubectl exec <pod> -- sh -c 'echo x > /data/f'    # sh: can't create /data/f: Read-only file system
```

WHY `readOnly: true` mounts the volume read-only inside the container; any write attempt fails with “Read-only file system.”

{L4.30}

> **Q:** Set a sizeLimit on an emptyDir and explain what happens if the container exceeds it.

DO

```
  volumes:
  - name: scratch
    emptyDir:
      sizeLimit: 100Mi
  # mount at /scratch
```

VERIFY

```
kubectl exec <pod> -- sh -c 'dd if=/dev/zero of=/scratch/big bs=1M count=200'
kubectl get pod <pod> -w                      # pod gets Evicted
```

```
kubectl describe pod <pod> | grep -i evict
```

WHY `sizeLimit` is a **soft cap enforced by the kubelet via eviction**: when usage exceeds it, the kubelet evicts the pod (it is not a hard filesystem quota).

{L4.31}

> **Q:** Create a generic Secret from literals for a DB username/password, then show the stored values are base64-encoded, not encrypted.

DO

```
kubectl create secret generic dbcreds --from-literal=username=admin --from-literal=password=s3cret
```

VERIFY

```
kubectl get secret dbcreds -o yaml                                  # data values are base64
kubectl get secret dbcreds -o jsonpath='{.data.password}'; echo     # czNjcmV0
kubectl get secret dbcreds -o jsonpath='{.data.password}' | base64 -d; echo   # s3cret
```

WHY Secret `data` is **base64-encoded, not encrypted** — anyone with read access (or unencrypted etcd access) can decode it. base64 is an encoding, not a security control.

{L4.32}

> **Q:** Create a Secret from files (--from-file) holding a certificate and key, and identify the resulting keys.

DO

```
printf 'cert-bytes' > cert.pem ; printf 'key-bytes' > key.pem
kubectl create secret generic tlsfiles --from-file=tls.crt=cert.pem --from-file=tls.key=key.pem
```

VERIFY

```
kubectl get secret tlsfiles -o jsonpath='{.data}'; echo    # keys: tls.crt, tls.key
```

WHY With `--from-file`, the **key defaults to the filename**; `key=path` renames it. Here `tls.crt=cert.pem` makes the key `tls.crt`. Without renaming you'd get keys `cert.pem`/`key.pem`.

{L4.33}

> **Q:** Create a kubernetes.io/tls typed Secret from a cert/key pair and explain where it's consumed.

DO

```
kubectl create secret tls mytls --cert=cert.pem --key=key.pem
```

VERIFY

```
kubectl get secret mytls -o jsonpath='{.type}'; echo       # kubernetes.io/tls
kubectl get secret mytls -o jsonpath='{.data}'; echo        # tls.crt, tls.key
```

WHY A `kubernetes.io/tls` Secret has fixed keys `tls.crt`/`tls.key`. It's consumed by **Ingress** resources for TLS termination (and by pods needing the cert/key pair).

{L4.34}

> **Q:** Mount a Secret as a volume and explain the security trade-offs versus env vars.

DO — volume way (preferred) as in {L4.26}; env way for comparison:

```
      env:
      - name: PASSWORD
        valueFrom:
          secretKeyRef: {name: dbcreds, key: password}
```

VERIFY

```
kubectl exec <pod> -- cat /etc/sec/password     # volume
kubectl exec <pod> -- printenv PASSWORD          # env
```

WHY (trade-offs) Volume: contents can auto-update on Secret change, are **not** shown in `kubectl describe pod`, aren't inherited by child processes, and can have restrictive file perms → safer. Env var: simpler for apps that read env, but the value appears in `describe`/`-o yaml`, leaks to child processes and crash dumps, and won't update without a restart.

{L4.35}

> **Q:** Use envFrom with a secretRef to load every key of a Secret as env vars at once.

DO

```
      envFrom:
      - secretRef:
          name: dbcreds
```

VERIFY

```
kubectl exec <pod> -- env | grep -E 'username|password'
```

WHY `envFrom.secretRef` injects **all** keys of the Secret as environment variables in one shot (key name = env var name). `configMapRef` does the same for a ConfigMap.

{L4.36}

> **Q:** Create a docker-registry (image-pull) Secret and reference it via imagePullSecrets; explain when it's required.

DO

```
kubectl create secret docker-registry regcred \
  --docker-server=quay.io --docker-username=u --docker-password=p
spec:
  imagePullSecrets:
  - name: regcred
  containers:
  - name: app
    image: quay.io/yourorg/private:1.0
```

VERIFY

```
kubectl get secret regcred -o jsonpath='{.type}'; echo     # kubernetes.io/dockerconfigjson
kubectl describe pod <pod> | grep -i pull                   # no ImagePullBackOff
```

WHY Required when pulling from a **private registry**; without it the pod fails with `ImagePullBackOff`. Public images don't need it; authenticated DockerHub also uses it to raise pull limits.

{L4.37}

> **Q:** Create a Secret with several keys and selectively mount only one of them into a pod.

DO

```
kubectl create secret generic multi --from-literal=a=1 --from-literal=b=2 --from-literal=c=3
  volumes:
  - name: sec
    secret:
      secretName: multi
      items:
      - key: b
        path: only-b.txt
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","sleep 3600"]
    volumeMounts: [{name: sec, mountPath: /etc/sec}]
```

VERIFY

```
kubectl exec <pod> -- ls /etc/sec       # only-b.txt (a and c are NOT mounted)
kubectl exec <pod> -- cat /etc/sec/only-b.txt   # 2
```

WHY The `items` list projects **only** the listed key(s); other keys in the Secret are excluded from the mount.

{L4.38}

> **Q:** Create a ClusterIP Service for a Deployment and resolve it by DNS from another pod.

DO

```
kubectl expose deployment web --port=80      # ClusterIP by default
```

VERIFY

```
kubectl get svc web
kubectl run tmp --rm -it --image=busybox -- nslookup web.default.svc.cluster.local
kubectl run tmp --rm -it --image=busybox -- wget -qO- web:80   # short name works same-ns
```

WHY ClusterIP gives a stable virtual IP reachable inside the cluster; CoreDNS resolves `<svc>.<ns>.svc.cluster.local` to it. Same-namespace callers can use the short name `web`.

{L4.39}

> **Q:** Create a NodePort Service and reach it via minikube service <svc> --url and $(minikube ip):<nodePort>.

DO

```
kubectl expose deployment web --type=NodePort --port=80
```

VERIFY

```
kubectl get svc web                          # note the 3xxxx nodePort
minikube service web --url
curl $(minikube ip):$(kubectl get svc web -o jsonpath='{.spec.ports[0].nodePort}')
```

WHY NodePort opens a port (30000–32767) on every node that forwards to the Service. `minikube service` prints/opens the reachable URL.

{L4.40}

> **Q:** Create a LoadBalancer Service, explain why EXTERNAL-IP is <pending> on minikube, and obtain access with minikube tunnel.

DO

```
kubectl expose deployment web --type=LoadBalancer --port=80
kubectl get svc web                          # EXTERNAL-IP <pending>
# in a SEPARATE terminal (needs sudo):
minikube tunnel
# back in the first terminal:
kubectl get svc web                          # EXTERNAL-IP now assigned
curl <external-ip>:80
```

WHY minikube has **no cloud load-balancer provider** to allocate an external IP, so it stays `<pending>`. `minikube tunnel` creates a route on the host that fulfils the LoadBalancer and assigns a reachable IP.

{L4.41}

> **Q:** Create a headless Service (clusterIP: None) for a StatefulSet and show it returns per-pod A records instead of a single VIP.

DO — the headless `redis` Service from {L4.16} (`clusterIP: None`).

VERIFY

```
kubectl get svc redis                         # CLUSTER-IP None
kubectl run tmp --rm -it --image=busybox -- nslookup redis.default.svc.cluster.local
# returns A records for redis-0/1/2 pod IPs (no single VIP)
```

WHY A headless Service has no ClusterIP, so DNS returns **one A record per ready pod** (`redis-0.redis…`, etc.) rather than a single virtual IP — letting clients address individual stable pods (needed for stateful peers).

{L4.42}

> **Q:** Inspect a Service's Endpoints/EndpointSlice and explain how they're populated from the pod selector.

DO / VERIFY

```
kubectl get endpoints web
kubectl get endpointslices -l kubernetes.io/service-name=web
kubectl describe endpoints web
```

WHY The endpoints controller watches pods that **match the Service's selector** and are **Ready**, and lists their `IP:port`. If no pods match or none are Ready, endpoints are empty and the Service returns nothing.

{L4.43}

> **Q:** Demonstrate cross-namespace access using the FQDN, and show the short name fails from another namespace.

DO / VERIFY

```
kubectl create namespace other
kubectl run tmp -n other --rm -it --image=busybox -- wget -qO- web.default.svc.cluster.local   # wor
kubectl run tmp -n other --rm -it --image=busybox -- nslookup web                               # fa
```

WHY Short names resolve only within the **caller's** namespace (via the DNS search domain). Reaching a Service in another namespace requires the **FQDN** `<svc>.<ns>.svc.cluster.local`.

{L4.44}

> **Q:** Compare kubectl port-forward to a Service versus to a Pod — explain the difference and when each fits.

DO / VERIFY

```
kubectl port-forward svc/web 8080:80          # via the Service port
# vs
kubectl port-forward pod/<podname> 8080:80    # to one specific pod
curl localhost:8080
```

WHY The `svc/` form targets the Service port and resolves to one backing pod; the `pod/` form pins a single pod (useful to debug one replica). Both are **local-machine tunnels** for testing and bypass real Service load-balancing.

{L4.45}

> **Q:** Explain the difference between a Service's port, targetPort, and nodePort.

SHOW

```
kubectl get svc web -o yaml | grep -A5 ports
```

WHY

- **port** — the Service's own port, what clients hit on the ClusterIP.

- **targetPort** — the **container** port traffic is forwarded to.

- **nodePort** — the port opened on every node (NodePort/LoadBalancer types only, 30000–32767).

{L4.46}

> **Q:** Verify connectivity to a ClusterIP Service from a throwaway debug pod with wget/nc.

DO / VERIFY

```
kubectl run tmp --rm -it --image=busybox -- sh
# inside the pod:
wget -qO- web.default.svc.cluster.local:80
nc -zv web 80
nslookup web
exit
```

WHY A throwaway busybox pod (`--rm -it`) tests in-cluster reachability of the Service; `wget`/`nc` confirm the port answers, `nslookup` confirms DNS.

{L4.47}

> **Q:** Create two Deployments and one Service that load-balances across both by sharing a common label; prove requests hit both.

DO (`shop.yaml` → `kubectl apply -f shop.yaml`)

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata: {name: shop-v1}
spec:
  replicas: 2
  selector: {matchLabels: {app: shop, ver: v1}}
  template:
    metadata: {labels: {app: shop, ver: v1}}
    spec:
      containers:
      - name: c
        image: hashicorp/http-echo
        args: ["-text=from-v1"]
        ports: [{containerPort: 5678}]
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: shop-v2}
spec:
  replicas: 2
  selector: {matchLabels: {app: shop, ver: v2}}
  template:
    metadata: {labels: {app: shop, ver: v2}}
    spec:
      containers:
      - name: c
        image: hashicorp/http-echo
        args: ["-text=from-v2"]
        ports: [{containerPort: 5678}]
---
apiVersion: v1
kind: Service
metadata: {name: shop}
spec:
  selector: {app: shop}            # matches BOTH deployments
  ports: [{port: 80, targetPort: 5678}]
```

VERIFY

```
kubectl get endpoints shop        # 4 pod IPs (both deployments)
kubectl run tmp --rm -it --image=busybox -- sh -c 'for i in $(seq 8); do wget -qO- shop; done'
# output mixes "from-v1" and "from-v2"
```

WHY The Service selects only the common label `app=shop`, so its endpoints include pods from both Deployments → it load-balances across all of them.

{L4.48}

> **Q:** Systematically diagnose why a pod can't reach a Service: DNS → endpoints → selector labels → readiness → port.

DO (in order)

```
kubectl run tmp --rm -it --image=busybox -- nslookup web                        # 1 DNS resolves?
kubectl get endpoints web                                                        # 2 endpoints popu
kubectl get svc web -o jsonpath='{.spec.selector}'; echo                         # 3 selector...
kubectl get pods --show-labels                                                   #   ...vs pod labe
kubectl get pods -l app=web -o jsonpath='{range .items[*]}{.metadata.name}{" "}{.status.conditions[?
kubectl get svc web -o jsonpath='{.spec.ports}'; echo                            # 5 port/targetPor
```

WHY Walk the path in order; the **first step that fails is the culprit** (NXDOMAIN → DNS; empty endpoints → selector/label mismatch or no Ready pods; wrong targetPort → connection refused).

{L4.49}

> **Q:** Expose a service with a NodePort and verify it works.

DO

```
kubectl expose deployment web --type=NodePort --port=80 --name=web-np
```

VERIFY

```
kubectl get svc web-np
curl $(minikube ip):$(kubectl get svc web-np -o jsonpath='{.spec.ports[0].nodePort}')
# or:
minikube service web-np --url
```

WHY NodePort makes the Service reachable on `<nodeIP>:<nodePort>`; a successful `curl` confirms external reachability through the node.

{L4.50}

> **Q:** Add an HTTP livenessProbe hitting / on port 80 to an nginx Deployment and verify via kubectl describe pod.

DO — manifest fragment in the container:

```
livenessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 10
```

Imperative patch:

```
kubectl patch deployment web --type=json -p='[{"op":"add","path":"/spec/template/spec/containers/0/li
```

VERIFY

```
kubectl describe pod -l app=web | grep -A2 Liveness     # Liveness: http-get http://:80/ ...
```

WHY The kubelet performs an HTTP GET on `/:80`; a non-2xx/3xx (or no response) fails the probe and the kubelet **restarts the container**.

{L4.51}

> **Q:** Add a tcpSocket readiness probe to a redis container on port 6379 and verify it.

DO

```
readinessProbe:
  tcpSocket:
    port: 6379
  periodSeconds: 5
```

VERIFY

```
kubectl describe pod -l app=redis | grep -A2 Readiness
kubectl get pods -l app=redis        # READY 1/1 once the TCP check passes
```

WHY The kubelet opens a TCP connection to 6379; if it fails, the pod is marked **not Ready** and **removed from Service endpoints** (it is **not** restarted — that's the liveness probe's job).

{L4.52}

> **Q:** Configure a startup probe with a failureThreshold * periodSeconds budget for a ~2-minute boot and justify the numbers.

DO

```
startupProbe:
  httpGet:
    path: /healthz
    port: 80
```

```
    failureThreshold: 30
    periodSeconds: 5         # 30 * 5 = 150s budget (> 120s boot)
```

VERIFY

```
kubectl describe pod <pod> | grep -A2 Startup
```

WHY `failureThreshold × periodSeconds` must exceed the worst-case boot time: `30 × 5 = 150s > 120s`. While the startup probe is running, the liveness and readiness probes are **held off**, so a slow-booting app isn't killed prematurely.

{L4.53}

> **Q:** Explain the default behavior when no probes are defined at all.

WHY With no probes, Kubernetes treats the container as **healthy as soon as its process starts**: it's marked Ready immediately and is only restarted if the **process itself exits/crashes**. There is no application-level health checking, so a process that is hung but still running will continue to receive traffic.

::: LO5 — Application shipping: find & fix the mistake ::: lo5

> Each: the broken command/file → the **FIX** (corrected, runnable) → **WHY**.

{L5.1}

> **Q:** podman run -d --name web -p 80:8080 docker.io/library/nginx (nginx listens on 80; the site isn't reachable on the host port you expected — what's reversed?)

FIX

```
podman run -d --name web -p 80:80 docker.io/library/nginx
curl localhost:80
```

WHY `-p host:container`. The original maps host **80** → **container 8080**, but nginx listens on **80** inside, so nothing answers. Map to the right container port.

{L5.2}

> **Q:** podman run -d --name db -e MYSQL_ROOT_PASSWORD docker.io/library/mysql (the container exits during init — inspect podman logs.)

FIX

```
podman run -d --name db -e MYSQL_ROOT_PASSWORD=secret docker.io/library/mysql
podman logs db          # init proceeds
```

WHY `-e MYSQL_ROOT_PASSWORD` with no value passes it **empty**; MySQL's entrypoint aborts because it has no root password. Give it a value (or `-e MYSQL_ALLOW_EMPTY_PASSWORD=1`

/ `MYSQL_RANDOM_ROOT_PASSWORD=1`).

{L5.3}

> **Q:** podman run -d --name app --network host -p 8080:80 docker.io/library/nginx (why is the -p flag effectively ignored here?)

FIX

```
# want port mapping? drop host networking:
podman run -d --name app -p 8080:80 docker.io/library/nginx
curl localhost:8080
# or keep host net and hit the real port directly:
# podman run -d --name app --network host docker.io/library/nginx ; curl localhost:80
```

WHY `--network host` shares the host's network namespace, so the container's ports are the host's ports. There's no separate namespace to map, so `-p` is **ignored** (podman prints a warning).

{L5.4}

> **Q:** podman run -d --name c1 docker.io/library/busybox (the container goes straight to Exited (0) — why, and how do you keep it running?)

FIX

```
podman run -d --name c1 docker.io/library/busybox sleep 1d     # or: sleep infinity
podman ps
```

WHY busybox's default command runs and immediately returns, so the container exits `0`. A container lives only as long as its PID 1 process — give it a long-running command.

{L5.5}

> **Q:** podman run --rm -d --name job docker.io/library/alpine echo hello (then podman logs job fails — explain the --rm + detached gotcha.)

FIX

```
podman run -d --name job docker.io/library/alpine echo hello
podman logs job
# or see output live without detaching:
podman run --rm docker.io/library/alpine echo hello
```

WHY `--rm` deletes the container the moment it exits. A detached `echo` finishes instantly, so the container (and its logs) are **gone** before `podman logs` runs.

{L5.6}

> **Q:** podman run -d -p 8080:80 --memory 8m docker.io/library/mysql (the database never becomes healthy — what limit is the problem?)

FIX

```
podman run -d -p 8080:3306 --memory 1g -e MYSQL_ROOT_PASSWORD=secret docker.io/library/mysql
```

WHY 8 MB is far below MySQL's footprint; it's OOM-killed during initialization and never becomes healthy. Raise `--memory` (≥512m, 1g is safe). (Also note MySQL needs a root password and listens on 3306.)

{L5.7}

> **Q:** two alpine containers, podman exec a ping b — name resolution fails on the default network; why, and how do you fix it?

FIX

```
podman network create mynet
podman run -d --name a --network mynet docker.io/library/alpine sleep 1d
podman run -d --name b --network mynet docker.io/library/alpine sleep 1d
podman exec a ping -c1 b
```

WHY Podman's **default** (rootless) network provides no inter-container DNS. A **user-defined network** enables name-based resolution between attached containers.

{L5.8}

> **Q:** podman run -d --name web -v ./html:/usr/share/nginx/html docker.io/library/nginx (files aren't visible / SELinux denies access — what mount flag is missing?)

FIX

```
podman run -d --name web -v ./html:/usr/share/nginx/html:Z docker.io/library/nginx
# (use an absolute path if a relative one isn't resolved)
```

WHY On SELinux hosts the bind mount needs a relabel flag: **`:Z`** (private label for this container) or `:z` (shared). Without it SELinux blocks the container from reading the host files.

{L5.10}

> **Q:** FROM debian:12 / RUN apt-get install -y nginx (the build fails to find the package — what's missing before install?)

FIX

```
FROM debian:12
RUN apt-get update && apt-get install -y nginx
```

WHY The base image ships **no package index**; you must `apt-get update` first (same `RUN`) so apt can find `nginx`.

{L5.11}

> **Q:** ... CMD python app.py (the app can't be stopped cleanly — shell vs exec form?)

FIX

```
CMD ["python", "app.py"]
```

WHY Shell form `CMD python app.py` runs as `/bin/sh -c "…"`, so **/bin/sh is PID 1** and doesn't forward `SIGTERM` to python → the app can't shut down gracefully. **Exec form** (JSON array)

makes the app PID 1 and receives signals directly.

{L5.12}

> **Q:** FROM node:20 / COPY . /app / WORKDIR /app / RUN npm install (every source change triggers a full npm install — reorder for layer caching.)

FIX

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
```

WHY Copy the dependency manifests **before** `npm install` so that layer is cached and only re-runs when `package*.json` changes. Copying all source first invalidates the install layer on every edit.

{L5.13}

> **Q:** FROM alpine:3.20 / ENV PATH=/app/bin / RUN apk add --no-cache curl (curl/sh aren't found — what did setting PATH break?)

FIX

```
FROM alpine:3.20
ENV PATH="/app/bin:${PATH}"
RUN apk add --no-cache curl
```

WHY `ENV PATH=/app/bin` **overwrites** PATH entirely, dropping `/usr/bin:/bin:/usr/sbin:/sbin`, so system binaries (sh, curl, apk) can't be found. Append to the existing PATH instead of replacing it.

{L5.14/15}

> **Q:** FROM ubuntu:24.04 / EXPOSE 8080 / CMD ["python3","-m","http.server","3000"] — you map -p 8080:8080 but nothing answers; what mismatch is there, and what does EXPOSE actually do?

FIX — make the listening port match what you publish:

```
FROM ubuntu:24.04
EXPOSE 8080
CMD ["python3", "-m", "http.server", "8080"]
podman run -d -p 8080:8080 <image>      # now answers
# (alternatively keep 3000 and map -p 8080:3000)
```

WHY The server listens on **3000** but you published **8080→8080**, so nothing is behind 8080. `EXPOSE` is **documentation/metadata only** — it does **not** publish or open any port; `-p` does the actual publishing, and the app must listen on the targeted container port.

{L5.16}

> **Q:** FROM debian:12 / RUN apt-get update && apt-get install -y build-essential (the image is huge — how do you cut the size in the same layer?)

FIX

```
FROM debian:12
RUN apt-get update \
 && apt-get install -y --no-install-recommends build-essential \
 && rm -rf /var/lib/apt/lists/*
```

WHY Clean the apt cache **in the same `RUN` layer** (and skip recommended extras). Removing it in a later layer wouldn't shrink the image, because each layer is additive.

{L5.17}

> **Q:** FROM python:3.11 / USER appuser / COPY requirements.txt … / RUN pip install … (permission denied — what's wrong with the USER/COPY ordering and ownership?)

FIX

```
FROM python:3.11
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
RUN useradd -m appuser && chown -R appuser /app
USER appuser
COPY . .
```

WHY Switching to a non-root `USER` **before** copying/installing means files are root-owned and the user can't write to system paths → permission denied. Do privileged work (install) first, set ownership, then drop to `USER appuser` last.

{L5.18}

> **Q:** FROM golang:1.22 … RUN go build … CMD ["/src/app"] (the final image is ~1 GB — how would a multi-stage build fix it?)

FIX

```
FROM golang:1.22 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /app .

FROM gcr.io/distroless/base-debian12
COPY --from=build /app /app
CMD ["/app"]
```

WHY A multi-stage build compiles in the heavy `golang` image but copies **only the resulting binary** into a tiny runtime base (distroless/alpine/scratch), dropping the ~1 GB toolchain from the final image.

{L5.19}

> **Q:** A pod is stuck Pending. Diagnose with kubectl describe pod and identify the reason from the events.

DO

```
kubectl describe pod <pod> | sed -n '/Events/,$p'
```

Read the event and act: `Insufficient cpu/memory` → lower `resources.requests` or add a node; `didn't match node selector/affinity` → label the node / fix selector; `untolerated taint` → add a toleration; `unbound PersistentVolumeClaim` → create the PVC.

{L5.20}

> **Q:** Remove a couple of spaces from a deployment and try to deploy it. Identify the cause in the error messages and fix it.

DO

```
kubectl apply -f broken.yaml
# error: did not find expected key  /  mapping values are not allowed in this context
kubectl apply -f broken.yaml --dry-run=client    # validate without applying
```

FIX/WHY YAML is indentation-sensitive; a misplaced space breaks the mapping structure. Restore consistent **2-space** indentation under the right parent key, then re-apply.

{L5.21}

> **Q:** A pod is in ImagePullBackOff. Diagnose the cause and fix it.

DO

```
kubectl describe pod <pod> | grep -A6 Events     # shows the exact pull error
```

FIX/WHY Causes: image **typo**, **missing/wrong tag**, or a **private registry without a pull secret**. Correct the image/tag, or `kubectl create secret docker-registry …` + `imagePullSecrets`, then re-deploy.

{L5.22}

> **Q:** A pod is in CrashLoopBackOff. Use kubectl logs --previous to read the last crash and fix the root cause.

DO

```
kubectl logs <pod> --previous
kubectl describe pod <pod> | grep -A3 'Last State'
```

FIX/WHY The container starts then exits repeatedly; `--previous` shows the **last crashed instance's** output (bad command, missing env/config, failed dependency). BackOff is the kubelet's increasing restart delay. Fix the root cause and redeploy.

{L5.23}

> **Q:** A pod's last state shows OOMKilled. Confirm it, then fix it.

DO

```
kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'; echo
kubectl describe pod <pod> | grep -i oom
```

FIX

```
kubectl set resources deployment web --limits=memory=512Mi
```

WHY The container exceeded its memory **limit** and was killed. Raise `limits.memory` or fix the app's memory usage.

{L5.24}

> **Q:** A Deployment's pods never become Ready. Trace it to a failing readiness probe and correct it.

DO

```
kubectl describe pod <pod> | grep -A4 Readiness     # shows probe failures
kubectl get pod <pod> -o jsonpath='{.status.conditions}'; echo
```

FIX/WHY A failing readiness probe keeps the pod out of endpoints (`READY 0/1`). Correct the probe's `path`/`port`/`initialDelaySeconds`, or fix the app endpoint it checks.

{L5.25}

> **Q:** A Service returns nothing. Diagnose a Service/pod label mismatch using kubectl get endpoints.

DO

```
kubectl get endpoints <svc>                              # empty = no matching/ready pods
kubectl get svc <svc> -o jsonpath='{.spec.selector}'; echo
kubectl get pods --show-labels
```

FIX/WHY Empty endpoints mean the Service **selector** doesn't match any pod labels (or pods aren't Ready). Align the selector with the pod labels (or fix the labels).

{L5.26}

> **Q:** spec.selector.matchLabels doesn't match spec.template.metadata.labels — explain the error and fix it.

DO

```
kubectl apply -f bad.yaml
# error: Deployment.apps "x" is invalid: spec.template.metadata.labels: Invalid value ...
#        `selector` does not match template `labels`
```

FIX/WHY A Deployment's `selector.matchLabels` must be a subset of the pod template labels. Make `template.metadata.labels` include every key/value in the selector.

{L5.27}

> **Q:** resources.requests exceed any node's capacity — explain why the pod is Pending and fix it.

DO

```
kubectl describe pod <pod>    # 0/N nodes available: Insufficient cpu/memory
kubectl describe nodes | grep -A5 Allocatable
```

FIX/WHY No node can satisfy the request, so the scheduler leaves the pod **Pending**. Lower `resources.requests` to fit a node, or add a larger node.

{L5.28}

> **Q:** A pod mounts a PVC that doesn't exist — diagnose the FailedMount/Pending and create the PVC.

DO

```
kubectl describe pod <pod>    # persistentvolumeclaim "data" not found
```

FIX

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: data}
spec:
  accessModes: [ReadWriteOnce]
  resources: {requests: {storage: 1Gi}}
kubectl apply -f pvc.yaml
```

WHY A pod referencing a missing PVC can't mount its volume and stays Pending. Create the PVC (a default StorageClass dynamically provisions the PV).

{L5.29}

> **Q:** An ubuntu container with no long-running command shows Completed/CrashLoopBackOff. Explain why and add a proper command.

FIX

```
          command: ["sleep", "infinity"]
```

WHY The `ubuntu` image has no foreground process, so the container exits immediately → `Completed` (with `restartPolicy: Never`) or `CrashLoopBackOff` (with `Always`). Give it a long-running command (or the real app).

{L5.30}

> **Q:** Use kubectl get events --sort-by=.lastTimestamp to triage everything that happened to a pod over time.

DO

```
kubectl get events --sort-by=.lastTimestamp
kubectl get events --sort-by=.lastTimestamp --field-selector involvedObject.name=<pod>
```

WHY Sorted events give a chronological timeline (scheduling, pulls, probe failures, OOM, restarts) for root-cause triage.

{L5.31}

> **Q:** Use kubectl exec -it to get a shell in a pod and debug DNS with nslookup and cat /etc/resolv.conf.

DO

```
kubectl exec -it <pod> -- sh
cat /etc/resolv.conf            # nameserver = CoreDNS, search domains
nslookup kubernetes.default
nslookup <svc>
exit
```

WHY Confirms the pod's DNS config (CoreDNS server + search domains) and whether service names resolve.

{L5.32}

> **Q:** Use an ephemeral debug container to troubleshoot a no-shell/distroless container.

DO

```
kubectl debug -it <pod> --image=busybox --target=<container> -- sh
```

WHY Distroless/no-shell containers have no `sh` to exec into. `kubectl debug` attaches an **ephemeral container** that shares the target's process/network namespaces, giving you tools without modifying the pod.

{L5.33}

> **Q:** Spin up a throwaway kubectl run tmp pod to test connectivity to a Service from inside the cluster.

DO

```
kubectl run tmp --rm -it --image=busybox -- sh
wget -qO- <svc>:80
nc -zv <svc> 80
exit
```

WHY `--rm -it` gives a disposable in-cluster client to verify Service reachability and DNS, auto-deleted on exit.

{L5.34}

> **Q:** A node shows NotReady. List what you'd check and inspect it on minikube.

DO

```
kubectl get nodes
kubectl describe node <node> | grep -A8 Conditions    # Ready / MemoryPressure / DiskPressure
minikube logs | tail
minikube ssh -- 'systemctl status kubelet'            # kubelet health
```

WHY A NotReady node usually means the **kubelet** is down, **disk/memory pressure**, or a broken **CNI/network plugin**. The node conditions and kubelet logs point to which.

{L5.35}

> **Q:** A pod is stuck Terminating. Explain finalizers and the grace period, then force-delete it and state the risks.

DO

```
kubectl get pod <pod> -o jsonpath='{.metadata.finalizers}'; echo
kubectl delete pod <pod> --grace-period=0 --force
```

WHY Finalizers block deletion until their controller completes cleanup; the **grace period** is the window between SIGTERM and SIGKILL. Force-deleting removes the API object immediately, but

the container **may still be running on the node** → risk of an orphaned process and, for stateful apps, **data corruption / split-brain**.

{L5.36}

> **Q:** Wrong apiVersion/kind pairing (e.g. apps/v1 + Pod) — explain the validation error and correct it.

DO

```
kubectl apply -f bad.yaml
# error: no matches for kind "Pod" in version "apps/v1"
```

FIX/WHY A **Pod** is `apiVersion: v1`; **Deployment/ReplicaSet/StatefulSet/DaemonSet** are `apps/v1`. Match the kind to its correct API group/version.

{L5.37}

> **Q:** A pod fails with CreateContainerConfigError because a configMapKeyRef key is missing. Diagnose and fix.

DO

```
kubectl describe pod <pod>    # couldn't find key <X> in ConfigMap <cm>
```

FIX/WHY The pod references a ConfigMap key that doesn't exist. Add the key to the ConfigMap (`kubectl edit cm <cm>`) or correct the `configMapKeyRef.key` in the pod spec.

{L5.38}

> **Q:** A pod can't mount a Secret that lives in a different namespace. Explain namespace scoping and fix it.

DO / FIX

```
# copy the secret into the pod's namespace:
kubectl get secret <s> -n <src-ns> -o yaml \
  | sed 's/namespace: <src-ns>/namespace: <pod-ns>/' \
  | kubectl apply -n <pod-ns> -f -
```

WHY Secrets are **namespace-scoped**; a pod can only reference Secrets in its **own** namespace. There's no cross-namespace mount — the Secret must exist alongside the pod.

{L5.39}

> **Q:** A rollout is stuck with ProgressDeadlineExceeded. Find the cause and remediate.

DO

```
kubectl rollout status deployment/web
kubectl describe deployment web | grep -A3 Conditions    # ProgressDeadlineExceeded
kubectl get pods -l app=web                               # find the failing new pods
kubectl describe pod <new-pod>
```

FIX/WHY The new ReplicaSet didn't become available within `progressDeadlineSeconds` — usually a bad image, failing probe, or insufficient resources on the new pods. Fix that root cause

(rollout resumes automatically) or `kubectl rollout undo deployment/web`.

{L5.40}

> **Q:** Catch indentation/field typos before applying with --dry-run=server / --validate=true, then fix them.

DO

```
kubectl apply -f f.yaml --dry-run=server
# e.g. error: unknown field "spec.template.spec.containers[0].imagePullpolicy"
```

FIX/WHY Server-side dry-run validates against the real schema and catches typos like `imagePullpolicy` (correct: **imagePullPolicy**) and misnested fields **before** anything is applied. Correct the field name/nesting and re-apply.

{L5.41}

> **Q:** An app can't reach its database Service. Verify in order and identify the broken link.

DO (in order)

```
kubectl get pod <app>                                  # 1 Running?
kubectl get svc db                                     # 2 Service exists?
kubectl get endpoints db                               # 3 endpoints populated?
kubectl exec <app> -- nslookup db                      # 4 DNS resolves?
kubectl get svc db -o jsonpath='{.spec.ports}'; echo   # 5 correct port?
```

WHY Walk pod → Service → endpoints → DNS → port; the **first failing check** is the broken link (e.g. empty endpoints → selector/label mismatch; NXDOMAIN → DNS; wrong port → connection refused).

{L5.42}

> **Q:** Read a container's state/lastState and restartCount with kubectl get pod -o jsonpath to find the cause of repeated restarts.

DO

```
kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].restartCount}'; echo
kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'; echo
kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}'; ec
```

WHY A high `restartCount` plus `lastState.terminated.reason` (e.g. **OOMKilled**, **Error**) and the exit code pinpoint why the container keeps restarting.

{L5.43}

> **Q:** Compare OpenShift Route to Ingress.

ANSWER Both expose HTTP(S) services outside the cluster. **Ingress** is the Kubernetes-native, portable resource: you write Ingress rules, but you must install a separate **ingress controller** (nginx, Traefik, HAProxy) for them to work. **Route** is OpenShift's built-in object, backed by the platform's integrated **HAProxy router** — no controller to install, it auto-generates a hostname, and

it offers simple TLS modes (**edge / passthrough / re-encrypt**) out of the box. OpenShift supports both and reconciles Ingress objects into Routes. In short: **Route** is simpler and native to OpenShift but non-portable; **Ingress** is the standard, portable choice that needs a controller.

::: LO6 — Evaluate orchestration systems (theory) ::: lo6

> Written answers you can screenshot. Each takes a clear position and justifies it with trade-offs. Your professor flagged that **Docker Swarm** and **k3s** are also fair game — covered at the end.

{L6.1}

> **Q:** Analyze the networking model of Kubernetes versus Docker's network.

ANSWER Docker's default model is host-centric: each host has a **bridge** network with **NAT**, containers get private IPs behind the host, you publish ports manually with `-p`, and DNS-based name resolution only works on user-defined networks. Kubernetes instead mandates a **flat, cluster-wide network**: every pod gets its own routable IP and **any pod can reach any pod across nodes without NAT**, implemented by **CNI plugins** (Calico, Flannel, Cilium). On top of that, **Services** provide stable virtual IPs and load balancing, and **CoreDNS** gives service discovery by name. So Docker networking is simple and local; Kubernetes networking is an abstraction designed for multi-node scale, discovery, and self-healing.

{L6.2}

> **Q:** Evaluate Kubernetes storage abstraction (CSI, StorageClasses, dynamic provisioning) against Podman's storage plugins. Which is more flexible for stateful workloads, and why?

ANSWER Kubernetes abstracts storage through **CSI drivers**, **StorageClasses**, and **dynamic provisioning**: a pod's **PVC** requests storage and a PV is provisioned on demand from whatever backend the class points at (cloud disks, NFS, Ceph), and the volume follows the pod when it's rescheduled to another node. Podman's volumes/plugins are essentially **host-local** storage. For stateful workloads **Kubernetes is far more flexible**, because it provides portable, dynamic, multi-node, lifecycle-managed storage with access modes and reclaim policies — exactly what databases and other stateful services need — whereas Podman storage is tied to a single host.

{L6.3}

> **Q:** Evaluate the operator pattern (CRDs + controllers) for managing stateful services versus using only k8s manifests, in terms of control and effort.

ANSWER Plain manifests describe desired state but leave **day-2 operations** (backups, failover, version upgrades, scaling a clustered database safely) to humans or external scripts. An **Operator**

encodes that operational knowledge into a custom **controller** that continuously reconciles a **CRD**, automating those tasks. The trade-off is effort vs control: operators require building or adopting and maintaining a controller (higher upfront cost and a learning curve), but they give far more **automated control** and dramatically reduce ongoing operational toil for complex stateful services. For simple/stateless apps, manifests are enough; for production databases, an operator usually pays off.

{L6.4}

> **Q:** Assess the developer experience of plain Kubernetes (kubectl + manifests) versus OpenShift's Source-to-Image (S2I) and developer console. What does each optimize for?

ANSWER Plain Kubernetes optimizes for **flexibility and control**: you author Dockerfiles and YAML manifests and manage everything explicitly, which is powerful but has a steep learning curve and a lot of boilerplate. OpenShift's **S2I** and **developer console** optimize for **developer velocity**: a developer can point S2I at source code and get an image built and deployed without writing a Dockerfile or manifests, all through a guided web UI. In short, kubectl+manifests give maximum control at the cost of effort; S2I+console trade some control for speed, guardrails, and a gentler onboarding for developers.

{L6.5}

> **Q:** Critically evaluate migrating a workload from OpenShift to Kubernetes.

ANSWER Migrating off OpenShift means giving up its integrated extras and rebuilding equivalents: **Routes** become Ingress (+ an ingress controller), **DeploymentConfig** becomes Deployment, **S2I/BuildConfig** become an external CI pipeline, you lose the **integrated registry**, the stricter **SCC** security defaults must be reproduced with PodSecurity/admission controllers, and you must stand up your own **monitoring/logging/console**. The migration effort is therefore substantial. The upside is **portability, no vendor coupling, and lower licensing cost**. Critically, it's a trade of integrated, supported tooling for flexibility and self-managed effort — worth it if portability/cost matter more than turnkey tooling, and risky if the team relies heavily on OpenShift's built-ins.

{L6.6}

> **Q:** Evaluate running CI/CD build agents inside Kubernetes versus on dedicated VMs, considering isolation, autoscaling, and noisy-neighbor effects.

ANSWER Running agents (Jenkins agents, GitLab runners, Tekton) **in Kubernetes** gives ephemeral, **autoscaling** capacity — pods spin up per build and disappear after — with good bin-packing and lower idle cost. The downsides are weaker **isolation** (containers share the node kernel) and **noisy-neighbor** risk where a heavy build starves others, which you mitigate with resource requests/limits, quotas, and node pools. **Dedicated VMs** give strong isolation and predictable performance but waste resources when idle and scale slowly/manually. Net: Kubernetes wins on elasticity and cost-efficiency; VMs win when you need hard isolation (e.g. untrusted or security-sensitive builds).

{L6.7}

> **Q:** How does Kubernetes integrate with cloud environments and dynamic capacity provisioning?

ANSWER Kubernetes integrates with clouds through the **cloud-controller-manager**, which provisions cloud **LoadBalancers** and attaches cloud **block storage** to PVCs via CSI. For dynamic capacity it scales at two levels: the **Horizontal Pod Autoscaler** (and VPA) scales workloads by metrics, while the **Cluster Autoscaler** (or **Karpenter** on AWS) adds and removes **nodes** automatically based on unschedulable-pod pressure. Combined with managed services (EKS/GKE/AKS), this lets a cluster grow and shrink its infrastructure on demand, matching capacity to load.

{L6.8}

> **Q:** Critically evaluate the default security and hardening of OpenShift versus vanilla Kubernetes. Why do enterprises in regulated industries often choose OpenShift?

ANSWER OpenShift is **secure-by-default**: **Security Context Constraints (SCC)** force containers to run as **non-root** unless explicitly granted otherwise, it ships built-in **OAuth/RBAC**, an integrated registry, image provenance/scanning, and Red Hat-signed, supported images. Vanilla Kubernetes is **permissive by default** — you must add PodSecurity admission, RBAC policies, NetworkPolicies, and scanning yourself, and mistakes are easy. Regulated industries (finance, healthcare, government) often choose OpenShift because it provides a **certified, vendor-supported, hardened baseline** with a clear audit and compliance story out of the box, reducing both risk and the effort of proving controls to auditors.

{L6.9}

> **Q:** Critically evaluate the learning curve and required skill set of OpenShift versus plain Kubernetes. Does OpenShift's added abstraction help or hinder a team new to containers?

ANSWER Plain Kubernetes already has a **steep** learning curve (manifests, networking, RBAC, controllers, storage). OpenShift layers its own concepts on top — **SCC, Routes, BuildConfig/ImageStream, the `oc` CLI** — so there's *more* to learn for deep operations. However, its **web console and S2I lower the barrier** for the common case: a newcomer can build and deploy an app without mastering Dockerfiles or YAML. So for a team new to containers, the abstraction **helps initial productivity** and onboarding, but can **hinder** later when they must understand the OpenShift-specific machinery to debug or operate at depth. Net: helpful early, with added complexity to master eventually.

{L6.10}

> **Q:** Compare the resource footprint of full Kubernetes versus k3s for a small on-premise deployment. When is full Kubernetes overkill?

ANSWER Full Kubernetes runs a heavyweight control plane — **etcd plus multiple components** — needing more CPU/RAM and ongoing operational care, and is built for large-scale HA. **k3s** packages the whole thing into a single small binary (<100 MB), can use **SQLite** instead of etcd,

strips legacy/in-tree cloud drivers, and runs comfortably on a Raspberry Pi — while exposing the **same kubectl/API**. For a small on-premise/edge/single-team deployment, **full Kubernetes is overkill**: its footprint and operational overhead aren't justified when k3s delivers the same developer-facing API at a fraction of the cost. Reach for full Kubernetes when you need large-scale HA, many nodes, or deep cloud integration.

{L6.11}

> **Q:** Defend whether a small team should use Docker Compose on a single host or move straight to Kubernetes, considering complexity, scaling needs, and resilience.

ANSWER Docker Compose on a single host is trivial to learn and operate and is well suited to development and small, stable single-host production — but it offers **no multi-node scheduling, no self-healing across hosts, no autoscaling**, and is a single point of failure. **Kubernetes** provides resilience, horizontal scaling, and rolling updates, at the cost of significant complexity and operational overhead. For a small team with modest, predictable load and no high-availability requirement, **Compose is the defensible choice** — it ships product fastest with least overhead. The team should move to Kubernetes only once it genuinely needs **HA, elastic scaling, or multi-host resilience**, at which point Compose's limits become the bottleneck.

{L6.12}

> **Q:** Analyze vendor lock-in across Kubernetes (open source / CNCF) and OpenShift (Red Hat). How does the choice affect long-term flexibility?

ANSWER Vanilla Kubernetes is **CNCF-governed and open**, with a portable API that runs across clouds and distributions, so lock-in is **low** and long-term flexibility is high. **OpenShift** is Red Hat's opinionated distribution: it adds value (support, integrated tooling, hardening) but also **Red-Hat-specific constructs** (Routes, DeploymentConfig, SCC, subscriptions) and licensing cost, which create **some lock-in**. The choice is a trade-off: OpenShift buys convenience and vendor accountability now, at the price of reduced portability later; plain Kubernetes preserves flexibility but you assemble and support the platform yourself.

{L6.13}

> **Q:** Defend a recommendation of OpenShift over vanilla Kubernetes for a company that wants an integrated, vendor-supported platform with built-in developer and operations tooling.

ANSWER For a company that explicitly wants an **integrated, supported platform**, OpenShift is the right call: it delivers one vendor-supported product with **built-in CI/CD (S2I/Tekton pipelines), an integrated registry, monitoring and logging, a developer console, hardened secure-by-default settings, and enterprise SLAs**. That means far less integration work than assembling these from separate open-source projects, faster developer onboarding, and a single accountable vendor for support and security patches. The trade-offs — licensing cost and some lock-in — are exactly what this company has chosen to accept in exchange for turnkey, supported tooling, so the recommendation is well justified.

{L6.14}

> **Q:** Compare storage provisioning between Kubernetes and regular VM workloads.

ANSWER With **VM workloads**, storage is provisioned manually and statically: an admin attaches disks/LUNs to a VM, and that storage is tied to the VM's lifecycle. **Kubernetes** makes storage **declarative and dynamic**: a developer's **PVC** requests capacity from a **StorageClass**, a **CSI** driver provisions a PV automatically, and the volume **follows the pod** when it's rescheduled to another node. So Kubernetes decouples the storage *request* from the *provisioning* and offers self-service, portability, and automated lifecycle management, whereas VM storage is admin-driven, host-bound, and manual.

{L6.15}

> **Q:** A startup with two engineers must ship a containerized product quickly and cheaply. Recommend a container solution and justify your choice on cost and operational overhead.

ANSWER Recommend a **managed PaaS / managed container service** (or **Docker Compose / k3s** if self-hosting), **not** self-managed full Kubernetes. With only two engineers, the priority is shipping product, not running a control plane. A managed platform (Cloud Run, Render, Fly, App Runner, or a small managed cluster) removes cluster upgrades, patching, and HA of the control plane, so the team pays mainly for what they run and spends near-zero time on ops. Full self-managed Kubernetes would impose **operational overhead that dwarfs a two-person team** and slow delivery — the opposite of "quick and cheap." If they self-host, Compose or k3s on one host keeps cost and overhead minimal.

{L6.16}

> **Q:** Compare Docker and Podman in terms of architecture (daemon vs daemonless) and rootless security. Why might a security-conscious team prefer Podman?

ANSWER Docker uses a central **daemon (dockerd) that traditionally runs as root**, which is a single point of failure and a broad attack surface — the Docker socket effectively grants root. **Podman is daemonless**: it forks containers directly as child processes (via conmon/runc) with no long-running privileged daemon, and it runs **rootless by default**, mapping the container root to an unprivileged host user. It's CLI-compatible with Docker and integrates well with **systemd and SELinux**. A security-conscious team prefers Podman because removing the root daemon and running rootless **shrinks the attack surface and improves isolation**, with no privileged background process to compromise.

{L6.17}

> **Q:** Compare the day-2 operational overhead (cluster upgrades, scaling nodes, backups) of self-managed Kubernetes versus a managed Kubernetes service.

ANSWER Self-managed Kubernetes puts all day-2 work on you: control-plane and node **upgrades**, **etcd backups** and restore testing, **node scaling**, OS/security **patching**, and ensuring control-plane **HA** — significant, specialized, ongoing effort. A **managed service**

(EKS/GKE/AKS/ARO) offloads the control plane: the provider runs, upgrades, and keeps it available, and often automates node scaling and backups, leaving you to manage mainly your workloads and worker nodes. The trade-off is some loss of control and a service fee in exchange for **dramatically lower operational overhead** — usually the better choice unless you have strong reasons (compliance, cost at scale, customization) to self-manage.

{L6.18}

> **Q:** A media-streaming company expects spiky, unpredictable global traffic. Recommend a containerized solution. Elaborate your choice.

ANSWER Recommend **managed Kubernetes with autoscaling, deployed across multiple regions, fronted by a CDN**. Streaming traffic that is spiky and global needs **elastic, automatic scaling**: the **Horizontal Pod Autoscaler** scales pods to demand and the **Cluster Autoscaler/Karpenter** adds nodes when needed and removes them when traffic falls, controlling cost. Kubernetes' **self-healing** and rolling updates keep the service available during spikes and deploys, **multi-region** placement reduces latency and adds resilience, and a **CDN** offloads the heavy media delivery. A managed control plane keeps the ops burden manageable while the platform absorbs unpredictable load — exactly the elasticity this scenario demands.

{L6.19}

> **Q:** Defend whether a company that has outgrown Docker Swarm (or a single Compose host) should migrate to Kubernetes — weigh the migration effort against the long-term benefits.

ANSWER If the company has genuinely outgrown Swarm/Compose, **migrating to Kubernetes is justified**. The **effort** is real: rewriting Compose/Swarm definitions into Kubernetes manifests, learning the platform, and rebuilding CI/CD, networking, and storage. But the **long-term benefits** are substantial and durable: a vast **ecosystem** (Helm, operators, service mesh, observability), **autoscaling**, **self-healing**, multi-cloud portability, and by far the largest community and talent pool. Swarm is effectively in **maintenance mode** and Compose can't scale across hosts, so staying put caps growth. When scaling and resilience needs are real and lasting, the one-time migration cost pays off; if needs are modest, it may not be worth it yet.

{L6.20}

> **Q:** Would you choose Kubernetes as a solution for a microservices architecture? Focus on orchestration, resilience and ecosystem.

ANSWER Yes. Kubernetes is the de facto platform for microservices. For **orchestration** it gives each service independent deployment, scaling, and rolling updates, with **service discovery** and **load balancing** built in. For **resilience** it self-heals (restarts failed pods, reschedules off dead nodes), supports health probes, and enables zero-downtime rollouts and rollbacks. Its **ecosystem** is unmatched — Helm for packaging, **service meshes** (Istio/Linkerd) for traffic management and mTLS, and a rich observability stack — all of which microservices benefit from. The only caveat is complexity: for a mere handful of services it can be overkill, but for a real microservices architecture the orchestration, resilience, and ecosystem make Kubernetes the strong choice.

{L6.21}

> **Q:** Would you choose Kubernetes for enterprise-grade applications? Focus on scalability, community support and feature richness.

ANSWER Yes. For enterprise-grade applications Kubernetes offers proven **scalability** — horizontal pod and cluster autoscaling to thousands of nodes/pods — so it grows with demand. **Community support** is its biggest asset: CNCF governance, the largest contributor and vendor ecosystem, abundant documentation, and a deep talent pool, which de-risks long-term adoption. Its **feature richness** — RBAC, namespaces and quotas for multi-tenancy, rolling updates, autoscaling, and extensibility via **CRDs/operators** — covers enterprise needs, and managed offerings add vendor support and SLAs. Together these make Kubernetes a robust, future-proof default for enterprise workloads, provided the organization invests in the requisite operational skills.

{L6.22}

> **Q:** Would you choose OpenShift as a solution for a microservices architecture? Focus on orchestration, resilience and ecosystem.

ANSWER Yes, especially for enterprises that want microservices **plus** integrated tooling. OpenShift is Kubernetes underneath, so it inherits the same **orchestration** (independent scaling, rolling updates, discovery) and **resilience** (self-healing, health checks, zero-downtime deploys). On top it adds an **ecosystem** tuned for microservices: built-in **CI/CD pipelines (Tekton/S2I)**, **OpenShift Service Mesh** (Istio), an integrated registry, and a developer console — reducing the assembly work of wiring these together yourself. The trade-offs are licensing cost and some vendor lock-in. For an organization that values an integrated, supported microservices platform over DIY flexibility, OpenShift is a strong choice.

{L6.23}

> **Q:** Would you choose OpenShift for enterprise-grade applications? Focus on scalability, community support and feature richness.

ANSWER Yes, when the enterprise values **vendor support and an integrated, hardened platform**. OpenShift delivers Kubernetes' **scalability** with added enterprise features: **secure-by-default SCC**, integrated **CI/CD, registry, monitoring/logging**, a developer console, and **Red Hat enterprise SLAs** and certified, signed images. "Community support" here comes in two forms — the broad Kubernetes/CNCF community beneath it **and** Red Hat's commercial support and lifecycle guarantees — which is exactly what regulated, support-dependent enterprises want. The **feature richness** reduces integration effort and strengthens the compliance/audit story. The cost is licensing and some lock-in, but for enterprises that prioritize supportability, security, and turnkey tooling, OpenShift is well justified.

::: LO6 EXTRA — Swarm, k3s, runtimes & broader theory ::: lo6-extra

Docker Swarm (prof flagged). Docker's built-in orchestrator: `docker swarm init`, **services** (`docker service create`), **stacks** (deploy a Compose file), an **overlay network** for multi-host, **Raft** consensus among managers, and a built-in routing mesh + secrets. Pros: trivial setup, Compose-native, gentle learning curve, fine for small clusters. Cons: small ecosystem, limited autoscaling/extensibility, and it's largely in **maintenance mode** industry-wide. vs Kubernetes: Swarm = simplicity at small scale; Kubernetes = power, ecosystem, and resilience at higher complexity.

k3s (prof flagged). A CNCF-certified, lightweight Kubernetes (Rancher/SUSE) in a single <100 MB binary bundling the API server, controller-manager, scheduler, kubelet and containerd. It defaults to **SQLite** (or embedded etcd for HA), has **server** and **agent** roles, and bundles Traefik ingress, a service-load-balancer, and local-path storage. Same kubectl/API as upstream Kubernetes. Ideal for **edge, IoT, dev, and small on-prem**; peers are k0s and microk8s. Choose it when full Kubernetes is overkill (small footprint, low ops) and full Kubernetes when you need large-scale HA and deep cloud integration.

Container runtimes & OCI. Kubernetes talks to runtimes via the **CRI**; the **dockershim was removed in v1.24**, so clusters use **containerd** or **CRI-O** (OpenShift's default) directly, both built on **runc**. Docker images still run because everything follows **OCI** image/runtime specs — that standardization is what keeps images portable across Docker, Podman, containerd, and CRI-O.

Other handy talking points. Helm (templated charts) vs **Kustomize** (overlays) vs raw manifests for packaging; **GitOps** (ArgoCD/Flux) reconciling the cluster to Git for auditability and easy rollback; **service mesh** for mTLS/traffic control at many-service scale; **HPA** (pods) vs **Cluster Autoscaler** (nodes); **etcd** as the single source of truth that must be backed up; **RBAC**, **NetworkPolicy** (default-allow → zero-trust), and **Pod Security Standards** (privileged/baseline/restricted; OpenShift uses **SCC**); and **when *not* to use Kubernetes** — a single app, tiny team, or stable load is better served by Compose, a PaaS, or serverless.

End of full answers. Verify exact image tags / namespaces against the live exam wording. Run the LO4/LO5 recipes once in minikube/podman tonight so the commands are muscle memory — a practical exam rewards speed and correctness.